

Risk Management in Financial Portfolio



Simon Leung · 14 min read · Dec 9, 2024



6



1



calculate Value at Risk (VaR) and Conditional VaR (CVaR) using different methods (historical, parametric, and Monte Carlo simulation) using Python

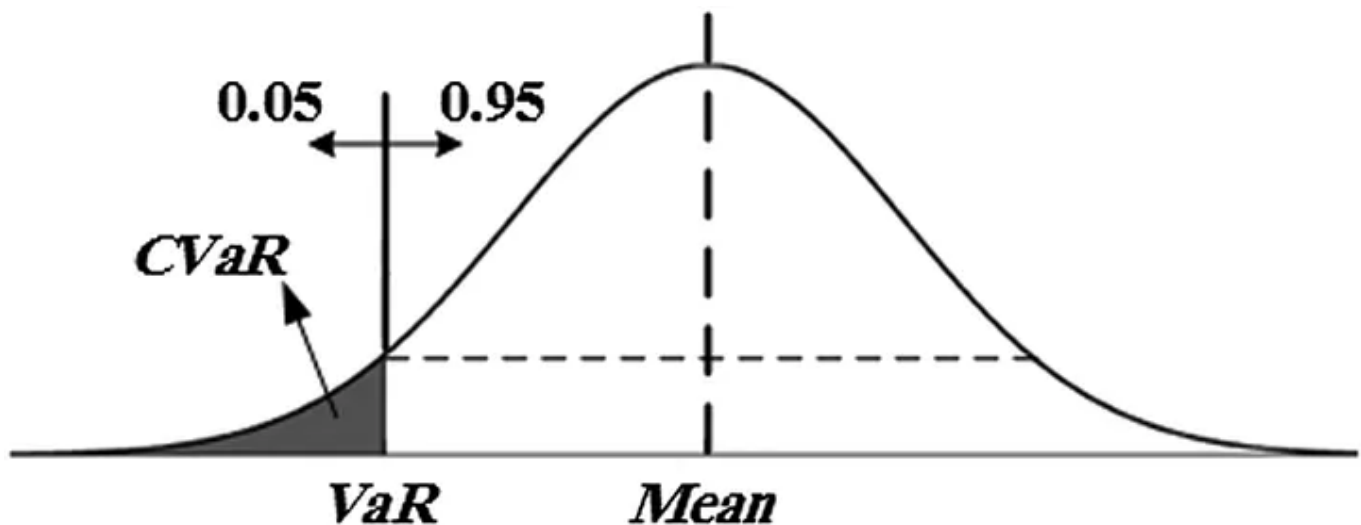


Image from https://www.researchgate.net/figure/Conditional-value-at-risk-CVaR-compared-with-VaR_fig4_326329658

Value at Risk (VaR) and Conditional Value at Risk (CVaR) are both risk measures used in financial risk management to quantify the potential loss in

the value of a portfolio over a defined period for a given confidence interval.

Value at Risk (VaR)

Definition: VaR is a statistical measure that quantifies the maximum potential loss in the value of a portfolio over a specified time period for a given confidence level. For example, if a portfolio has a one-day VaR of \$1 million at a 95% confidence level, there is a 5% chance that the portfolio will lose more than \$1 million in a single day.

Advantages:

1. **Simplicity:** VaR is relatively easy to understand and communicate.
2. **Widely Used:** It is a standard measure in the financial industry, making it easier to compare risk across different portfolios and institutions.
3. **Regulatory Acceptance:** Many regulatory bodies accept VaR as a risk measure for capital adequacy purposes.

Disadvantages:

1. **Tail Risk:** VaR does not provide information about the magnitude of losses beyond the VaR threshold. It ignores the tail risk.
2. **Sub-additivity:** VaR is not always sub-additive, meaning the VaR of a combined portfolio may be greater than the sum of the VaRs of its individual components.
3. **Assumptions:** VaR relies on assumptions about the distribution of returns, which may not hold true in all market conditions.

Conditional Value at Risk (CVaR)

Definition: CVaR, also known as Expected Shortfall (ES), is a risk measure that quantifies the expected loss given that the VaR threshold has been exceeded. It provides an average of the losses that occur beyond the VaR level. For example, if the one-day CVaR at a 95% confidence level is \$1.5 million, it means that, on average, the portfolio will lose \$1.5 million on days when the loss exceeds the VaR.

Advantages:

1. **Tail Risk:** CVaR provides information about the tail risk, giving a more comprehensive view of potential losses.
2. **Sub-additivity:** CVaR is sub-additive, meaning the CVaR of a combined portfolio will not be greater than the sum of the CVaRs of its individual components.
3. **Conservatism:** CVaR is generally considered more conservative and robust than VaR, especially in stress scenarios.

Disadvantages:

1. **Complexity:** CVaR is more complex to calculate and understand compared to VaR.
2. **Computational Intensity:** Calculating CVaR can be computationally intensive, especially for large portfolios.
3. **Less Widely Used:** While gaining traction, CVaR is still not as widely used or accepted as VaR, particularly in regulatory contexts.

Comparison

- **Information Provided:** VaR gives a threshold for potential losses, while CVaR provides an expected loss beyond that threshold.
- **Tail Risk:** CVaR accounts for tail risk, whereas VaR does not.
- **Sub-additivity:** CVaR is sub-additive, making it more suitable for portfolio risk management, while VaR is not always sub-additive.
- **Complexity:** VaR is simpler to calculate and understand, while CVaR is more complex but provides more detailed risk information.

In summary, while VaR is a useful and widely accepted risk measure, CVaR offers a more comprehensive view of potential losses, particularly in the tail of the distribution. The choice between VaR and CVaR often depends on the specific needs and regulatory requirements of the financial institution.

When calculating Value at Risk (VaR) and Conditional Value at Risk (CVaR), it is generally better to use the “**Adj. Close**” (**Adjusted Close**) price. Here’s why:

- **Adjusted Close Price:** This price adjusts for corporate actions such as dividends, stock splits, and other factors that can affect the stock price. Using the adjusted close price provides a more accurate reflection of the stock’s true value and performance over time.
- **Close Price:** The close price is the final price at which the stock traded during the regular trading hours on a given day. However, it does not account for dividends, stock splits, or other corporate actions.

Using the adjusted close price ensures that your calculations take into account the actual returns an investor would experience, including

dividends and other adjustments. This leads to more accurate risk metrics.

When calculating Value at Risk (VaR) and Conditional Value at Risk (CVaR), **log returns** are generally preferred over simple returns. Here's why:

1. **Normality Assumption:** Log returns are more likely to be normally distributed than simple returns, especially over shorter time periods. Many risk models, including VaR and CVaR, assume returns are normally distributed, so using log returns can produce more accurate results.
2. **Additivity:** Log returns are additive over time. This means that the total log return over multiple periods is simply the sum of the log returns over each period. This property is useful for aggregating returns over different time horizons.
3. **Compounding:** Log returns account for continuous compounding, which provides a more accurate measure of performance over time. Simple returns, on the other hand, assume discrete compounding.
4. **Negative Prices:** Simple returns can theoretically suggest negative prices, which isn't possible in reality. Log returns avoid this issue as they can't result in impossible price values.

When applying weights to calculate portfolio returns, the calculation should be done **after computing log returns** for individual assets. Here's why:

Correct Workflow:

1. Compute Log Returns for Each Asset:

$$r_i = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right)$$

2. Apply Portfolio Weights:

$$r_{\text{portfolio}} = \sum_{i=1}^n w_i \cdot r_i$$

Why After Computing Log Returns?

1. **Additivity of Log Returns:** Log returns are additive across assets when weighted, ensuring correct portfolio-level aggregation.
2. **Consistency with Portfolio Theory:** Portfolio metrics like expected return, variance, VaR, and CVaR are based on weighted individual asset returns.
3. **Mathematical Soundness:** Applying weights before computing log returns would distort the returns because log transformation is non-linear.

VaR and CVaR on the Left Side of the Curve

In most risk management contexts, VaR and CVaR are typically located on the left side of the return distribution curve because they measure potential

losses:

Left Side (Negative Returns/Losses):

- **VaR (Left Tail):** This indicates the maximum loss expected at a given confidence level (e.g., 95% or 99%). For example, if VaR at 95% confidence is -5%, it means there is a 5% chance the portfolio will lose more than 5% over a given period.
- **CVaR (Left Tail):** This measures the average loss beyond the VaR threshold, so it captures the severity of extreme losses. For example, if CVaR at 95% confidence is -7%, it means the average loss, given that losses exceed the VaR threshold, is 7%.

VaR and CVaR on the Right Side of the Curve

Although less common in risk management, you could conceptually examine VaR and CVaR on the right side of the curve, focusing on potential gains rather than losses:

Right Side (Positive Returns/Gains):

- **VaR (Right Tail):** This would indicate the minimum gain expected at a given confidence level. For example, if VaR at 95% confidence is 5%, it means there is a 95% chance the portfolio will gain at least 5% over a given period.
- **CVaR (Right Tail):** This would measure the average gain beyond the VaR threshold, capturing the severity of extreme gains. For example, if CVaR at 95% confidence is 7%, it means the average gain, given that gains exceed the VaR threshold, is 7%.

In practice, VaR and CVaR are primarily used to assess downside risk (left tail), focusing on potential losses to ensure risk is managed appropriately.

CVar > VaR: This is the more typical and expected scenario because CVaR, by definition, captures the average loss beyond the VaR threshold. Reasons for this include:

1. **Average Tail Loss:** CVaR measures the average loss in the tail beyond the VaR threshold, naturally leading to higher values since it incorporates more extreme outcomes.
2. **Risk Sensitivity:** CVaR is a more sensitive risk measure as it accounts for the severity of losses in the tail, while VaR only indicates the threshold.

Imagine the return distribution curve. VaR is a point on this curve (usually the 95th or 99th percentile), whereas CVaR is the average of losses beyond this point. Hence, CVaR tends to be higher since it includes the severe losses occurring in the tail beyond the VaR.

Example:

For a clearer illustration:

- **VaR at 95%:** If VaR at the 95% confidence level is -5%, it means there is a 5% chance the portfolio will lose more than 5% in a given period.
- **CVaR at 95%:** If CVaR at the 95% confidence level is -7%, it means the average loss, given that losses exceed the VaR threshold, is 7%.

In summary, **VaR** serves as a threshold indicator of potential loss, while **CVaR** provides a more comprehensive view of the risk by considering the severity of those extreme losses.

Implementation in Python

Arbitrarily pick four mutual funds from T. Rowe Price.

Disclaimer:

The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.

I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

```
# https://www.troweprice.com/personal-investing/tools/fund-research/PRCOX
# PRCOX U.S. Equity Research Fund
# https://www.troweprice.com/personal-investing/tools/fund-research/PREIX
# PREIX Equity Index 500 Fund
# https://www.troweprice.com/personal-investing/tools/fund-research/POMIX
# POMIX Total Equity Market Index Fund
# https://www.troweprice.com/personal-investing/tools/fund-research/PRDGX
# PRDGX Dividend Growth Fund
```

Download Historical Data and Plot it

```
import yfinance as yf
# Import data
stockList = ['POMIX', 'PRCOX', 'PRDGX', 'PREIX']

endDate = "2024-12-01"
startDate = "2020-01-01"

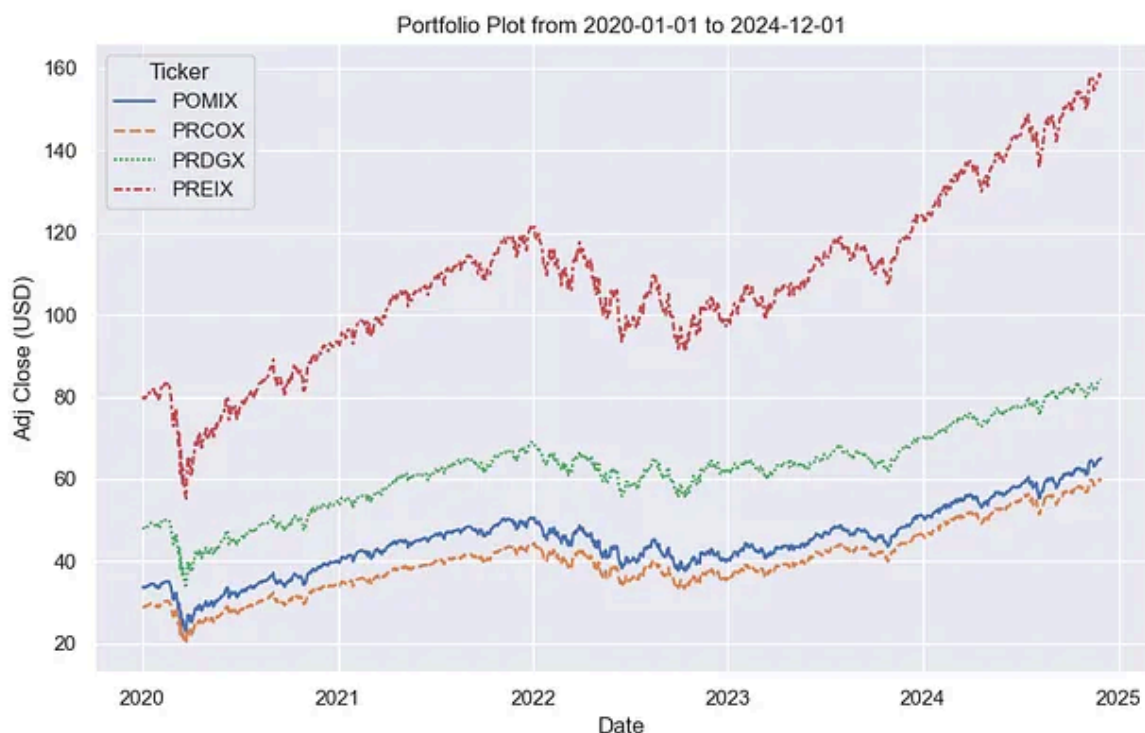
def getStockData(stocks, start, end):
    stockData = yf.download(stocks, start=start, end=end)
    stockData = stockData['Adj Close']
    return stockData

stockData = getStockData(stockList, start=startDate, end=endDate)
```

```
# Plotting multiple columns in one graph
plt.figure(figsize=(10, 6))
sns.lineplot(stockData)

# Adding titles and labels
plt.title(f'Portfolio Plot from {startDate} to {endDate}')
plt.xlabel('Date')
plt.ylabel('Adj Close (USD)')

# Show the plot
plt.savefig('Portfolio.png')
plt.show()
```



Calculate Log Returns and apply weights

```
def getReturnData(stockData):
    returns = np.log(stockData / stockData.shift(1))
    # Dropping NaN values resulting from the shift
    returns = returns.dropna()
    return returns

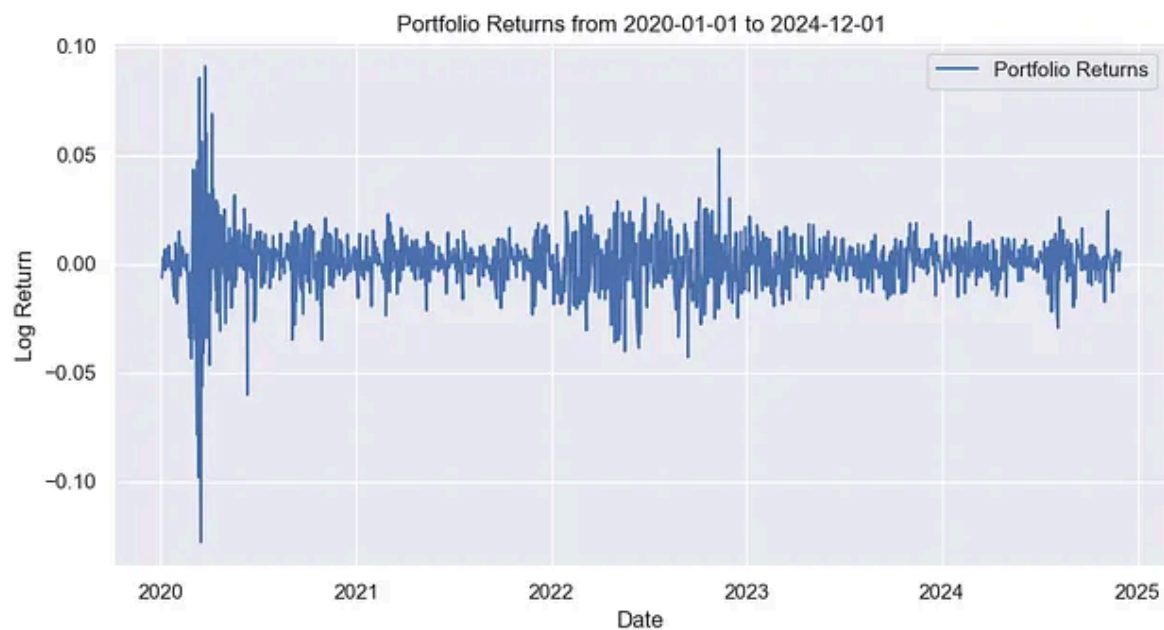
returns = getReturnData(stockData)

# Weights based on Overall Morningstar Rating (stars) As of 11/30/2024
# ['POMIX','PRCOX','PRDGX','PREIX']
weights = [3,5,3,4]
weights /= np.sum(weights)
print(weights)
returns['portfolio'] = returns.dot(weights)
```

```
# Plot the returns
plt.figure(figsize=(10, 5))
plt.plot(returns['portfolio'], label='Portfolio Returns')
```

```
plt.title(f'Portfolio Returns from {startDate} to {endDate}')
plt.xlabel('Date')
plt.ylabel('Log Return')
plt.legend()
plt.savefig('Portfolio Log Return.png')

# Show the plot
plt.show()
```



Calculate Maximum Drawdown

```
def calculate_max_drawdown(returns):
    """
    Calculate the maximum drawdown of a series of returns.

    Parameters:
    returns (list or numpy array): A series of periodic returns.

    Returns:
    float: The maximum drawdown as a percentage.
    """
    # Convert returns to numpy array if it's a list
    if isinstance(returns, list):
```

```
returns = np.array(returns)

# Calculate cumulative returns
cum_returns = (1 + returns).cumprod()

# Calculate running maximum
running_max = np.maximum.accumulate(cum_returns)

# Calculate drawdowns
drawdowns = (cum_returns - running_max) / running_max

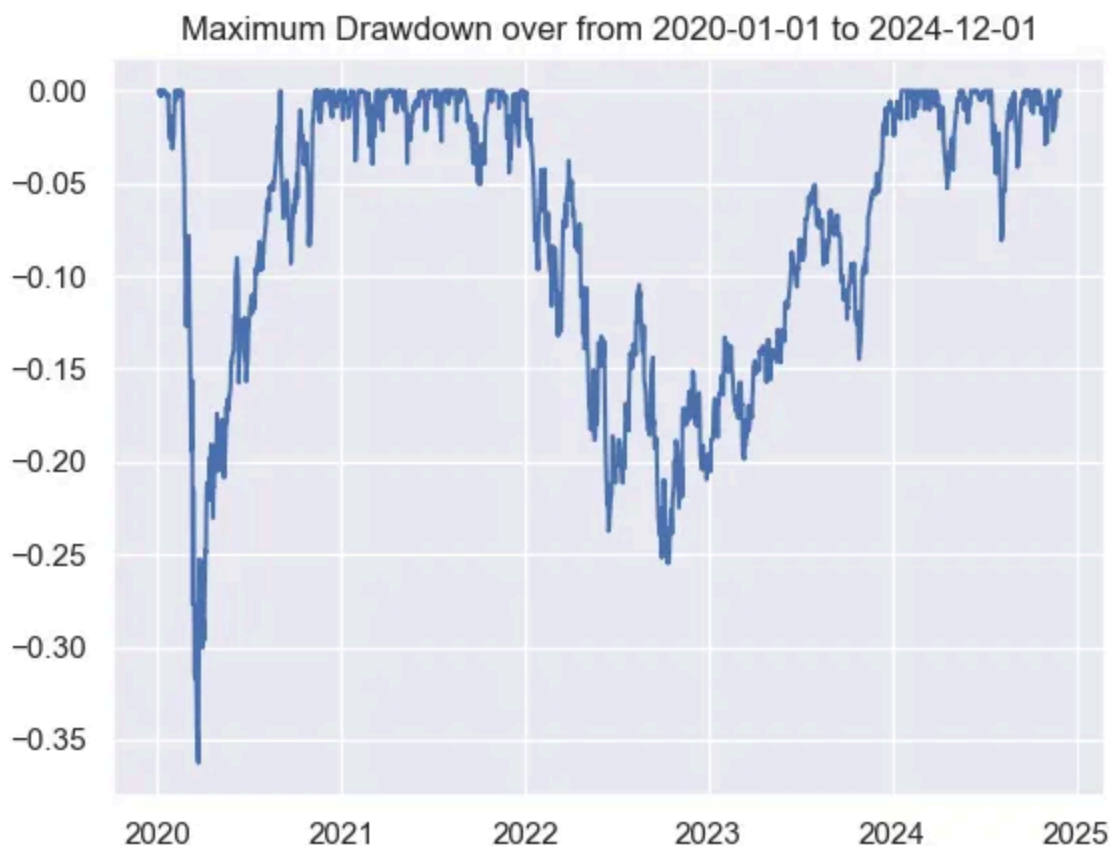
# Find the maximum drawdown
max_drawdown = np.min(drawdowns)

return max_drawdown, drawdowns

# Example usage
max_dd, dd = calculate_max_drawdown(returns['portfolio'])
print(f"Maximum Drawdown: {max_dd:.2%} over a given period from {startDate} to {endDate}")

plt.plot(dd)
plt.title(f'Maximum Drawdown over from {startDate} to {endDate}')
plt.savefig('Max Drawdown.png')

plt.show()
```



Maximum Drawdown: -36.25% over a given period from 2020-01-01 to 2024-12-01

Exam the distribution of portfolio returns

```
from scipy.stats import kstest, norm, t, laplace, cauchy

# Dictionary to hold the distributions and their corresponding fit methods
distributions = {
    'Normal': norm,
    'T-distribution': t,
    'Laplace': laplace,
    'Cauchy': cauchy
}
```

```

# Significance level
alpha = 0.05

# Function to perform KS test for each column in the DataFrame
def ks_test_dataframe(col, distributions, alpha=0.05):
    results = {}
    for dist_name, dist in distributions.items():
        # Fit the distribution to the sample and get the parameters
        params = dist.fit(col)
        # Perform the KS test using the fitted parameters
        ks_stat, p_value = kstest(col, dist.cdf, args=params)
        # Determine if we reject or fail to reject the null hypothesis
        if p_value > alpha:
            results[dist_name] = f'Fail to reject H0 (p-value: {p_value:.4f}), P
        else:
            results[dist_name] = f'Reject H0 (p-value: {p_value:.4f}), Parameter
    return results

# Perform the KS test for each column in the DataFrame
results = ks_test_dataframe(returns['portfolio'], distributions, alpha)

# Print the results
for dist, dist_results in results.items():
    print(f'Distribution: {dist} :\n {dist_results}')

```

```

Distribution: Normal :
    Reject H0 (p-value: 0.0000), Parameters: (0.0005404101613298844, 0.013315593915
Distribution: T-distribution :
    Fail to reject H0 (p-value: 0.8610), Parameters: (2.9052622131863126, 0.0009730
Distribution: Laplace :
    Fail to reject H0 (p-value: 0.3801), Parameters: (0.0008844100852984486, 0.0086
Distribution: Cauchy :
    Reject H0 (p-value: 0.0010), Parameters: (0.0009228485609285881, 0.005541625936

```

```

def fit_distributions(stock_data, column):
    plt.figure(figsize=(10, 6))

    daily_return = stock_data[column]

    # Fit a t-distribution to the daily return data
    t_params = t.fit(daily_return)
    dfree, t_mu, t_std = t_params

```

```
# Fit a Normal distribution to the daily return data
norm_params = norm.fit(daily_return)
norm_mu, norm_std = norm_params

# Fit a Laplace distribution to the daily return data
laplace_params = laplace.fit(daily_return)
laplace_mu, laplace_b = laplace_params

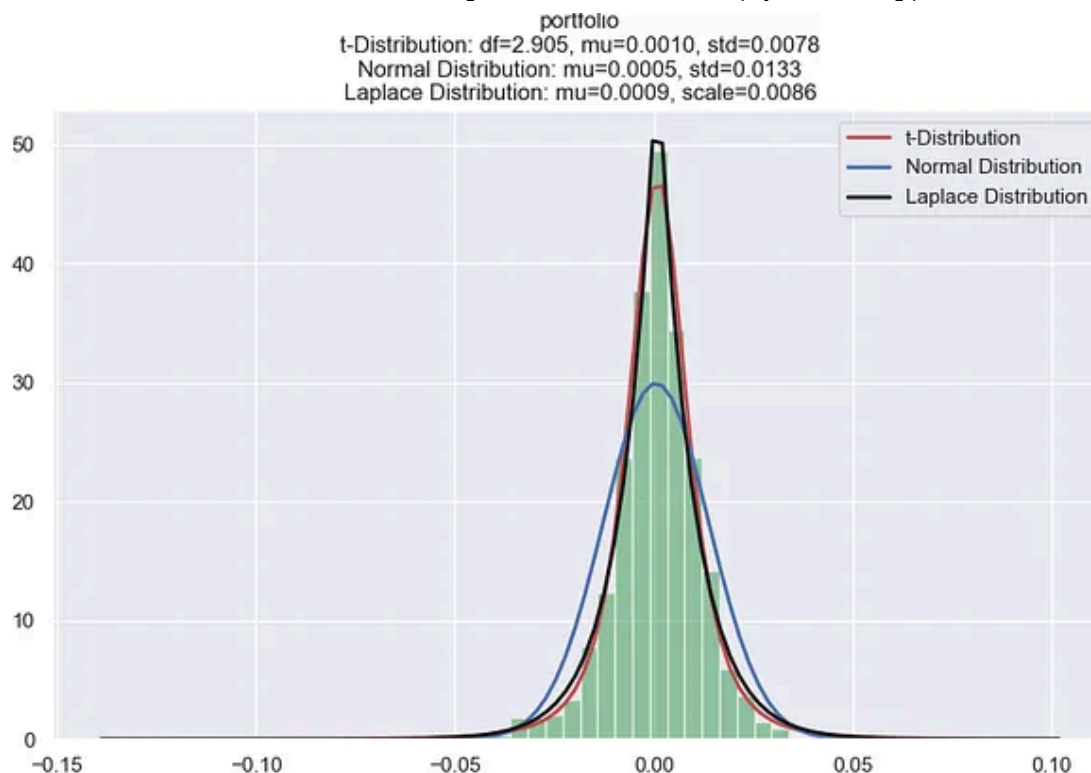
# Plot the histogram and the PDFs
plt.hist(daily_return, bins=50, density=True, alpha=0.6, color='g')
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)

t_pdf = t.pdf(x, dfree, t_mu, t_std)
norm_pdf = norm.pdf(x, norm_mu, norm_std)
laplace_pdf = laplace.pdf(x, laplace_mu, laplace_b)

plt.plot(x, t_pdf, 'r', linewidth=2, label='t-Distribution')
plt.plot(x, norm_pdf, 'b', linewidth=2, label='Normal Distribution')
plt.plot(x, laplace_pdf, 'k', linewidth=2, label='Laplace Distribution')

title = f"{column}\nt-Distribution: df={dfree:.3f}, mu={t_mu:.4f}, std={t_std:.4f}"
title += f"Normal Distribution: mu={norm_mu:.4f}, std={norm_std:.4f}\n"
title += f"Laplace Distribution: mu={laplace_mu:.4f}, scale={laplace_b:.4f}"
plt.title(title)
plt.legend()
plt.savefig('hist_plot.png')
plt.show()

fit_distributions(returns, 'portfolio')
```



The portfolio returns fit better with a t-distribution than a Laplace distribution. The t-distribution has a highest p-value.

Historical Approach: Calculate VaR and CVaR

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Function to calculate portfolio returns
#def portfolio_returns(asset_returns, weights):
#    return asset_returns.dot(weights)

# Function to scale returns for different time horizons
def scale_returns(portfolio_returns, time_horizon):
    """
```

Scale returns for different time horizons using the square-root rule.

:param returns: Array-like, historical returns

:param time_horizon: Number of periods to scale (e.g., 1 day, 5 days, 30 day

:return: Scaled returns

"""

return portfolio_returns * np.sqrt(time_horizon)

Function to calculate Historical VaR

```
def historical_var(portfolio_returns, confidence_level=0.05, investment=1_000_00
    scaled_returns = scale_returns(portfolio_returns, time_horizon)
    var_pct = np.percentile(scaled_returns, confidence_level * 100)
    var_amount = investment * abs(var_pct)
    return var_pct, var_amount
```

Function to calculate Historical CVaR

```
def historical_cvar(portfolio_returns, confidence_level=0.05, investment=1_000_0
    scaled_returns = scale_returns(portfolio_returns, time_horizon)
    var_pct, _ = historical_var(portfolio_returns, confidence_level, investment,
    cvar_pct = scaled_returns[scaled_returns <= var_pct].mean()
    cvar_amount = investment * abs(cvar_pct)
    return cvar_pct, cvar_amount
```

Function to display risk metrics summary

```
def risk_summary(portfolio_returns, confidence_level=0.05, investment=1_000_000,
    var_pct, var_amt = historical_var(portfolio_returns, confidence_level, inves
    cvar_pct, cvar_amt = historical_cvar(portfolio_returns, confidence_level, in
    mean_return = portfolio_returns.mean()
    std_dev = portfolio_returns.std()

    print(f"--- Risk Metrics Summary (Time Horizon: {time_horizon} Days, Investm
    print(f"Expected Return (Mean): {mean_return:.4f}")
    print(f"Standard Deviation (Volatility): {std_dev:.4f}")
    print(f"VaR ({confidence_level*100:.1f}%): {var_pct:.2%} | Monetary Loss: ${
    print(f"CVaR ({confidence_level*100:.1f}%): {cvar_pct:.2%} | Monetary Loss:
```

Function to plot portfolio returns with VaR and CVaR

```
def plot_var_cvar(portfolio_returns, confidence_level=0.05, investment=1_000_000
    scaled_returns = scale_returns(portfolio_returns, time_horizon)
    var_pct, var_amt = historical_var(portfolio_returns, confidence_level, inves
    cvar_pct, cvar_amt = historical_cvar(portfolio_returns, confidence_level, in

    plt.figure(figsize=(10, 6))
    sns.histplot(scaled_returns, bins=50, kde=True, color='skyblue')

    # Plot VaR and CVaR
    plt.axvline(var_pct, color='red', linestyle='--', label=f'VaR ({confidence_l
    plt.axvline(cvar_pct, color='purple', linestyle='--', label=f'CVaR ({confide

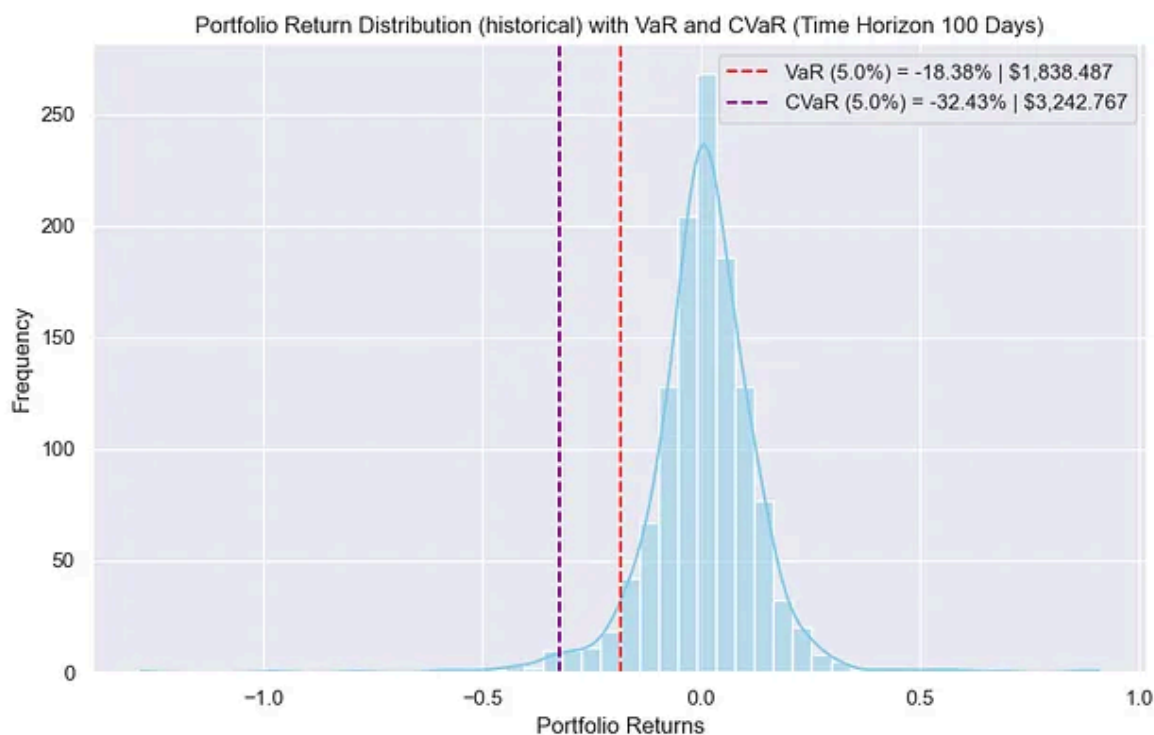
    plt.title(f'Portfolio Return Distribution (historical) with VaR and CVaR (Ti
    plt.xlabel('Portfolio Returns')
```

```
plt.ylabel('Frequency')
plt.legend()
plt.savefig('hist_var_cvar.png')
plt.show()
```

Example Calculation

```
initial_investment = 10000 # 10k
time_horizon = 100 # 100-day horizon
confidence_level = 0.05 # 5% for 95% confidence
```

```
risk_summary(returns['portfolio'], confidence_level, initial_investment, time_ho
plot_var_cvar(returns['portfolio'], confidence_level, initial_investment, time_h
```



```
--- Risk Metrics Summary (Time Horizon: 100 Days, Investment: $10,000) ---
Expected Return (Mean): 0.0005
Standard Deviation (Volatility): 0.0133
VaR (5.0%): -18.38% | Monetary Loss: $1,838.486
CVaR (5.0%): -32.43% | Monetary Loss: $3,242.765
```

Interpretation: There is a 5% chance that the portfolio will experience a loss greater than 18.38% (VaR) or Monetary Loss at \$1,838.486 if invested 10k over a 100 days period. If the portfolio experiences a loss greater than 1.838% (VaR), the average loss will be 32.43% (CVaR) or Monetary Loss at \$3,242.765 if invested 10k over a 100 days period.

Parametric Approach: Calculate VaR and CVaR

```
import numpy as np
import scipy.stats as stats

def parametric_var_cvar(portfolio_returns, confidence_level, time_horizon, initial_investment):
    # Parameters for the t-distribution
    df, mu, sigma = stats.t.fit(portfolio_returns)

    # VaR at the given confidence level
    var_value = stats.t.ppf(1 - confidence_level, df, mu, sigma) * np.sqrt(time_horizon)
    monetary_var = initial_investment * (var_value)
    var_percentage = var_value * 100 # VaR as a percentage

    # CVaR calculation
    cvar_value = (1 / (1 - confidence_level)) * stats.t.expect(
        lambda x: x,
        args=(df,), loc=mu, scale=sigma,
        lb=stats.t.ppf(confidence_level, df, mu, sigma)
    ) * np.sqrt(time_horizon)
    monetary_cvar = initial_investment * cvar_value
    cvar_percentage = cvar_value * 100 # CVaR as a percentage

    # Make CVaR value negative to reflect loss
    monetary_cvar = -abs(monetary_cvar)
    cvar_percentage = -abs(cvar_percentage)

    return monetary_var, var_percentage, monetary_cvar, cvar_percentage

confidence_level = 0.95 # 95% confidence level
time_horizon = 100 # 100 days
```

```

initial_investment = 10000 # Initial investment of $10,000

monetary_var, var_percentage, monetary_cvar, cvar_percentage = parametric_var_cv
    returns['portfolio'], confidence_level, time_horizon, initial_investment)

print(f"--- Risk Metrics Summary (Time Horizon: {time_horizon} Days, Investment:
print(f"VaR Monetary Loss over {time_horizon} days: ${-monetary_var:.2f} ({var_p
print(f"CVaR Monetary Loss over {time_horizon} days: ${-monetary_cvar:.2f} ({cva

# Parameters for the t-distribution
df, mu, sigma = stats.t.fit(returns['portfolio'])

t_values = stats.t.rvs(df, loc=mu, scale=sigma, size=10000) * np.sqrt(time_horiz
plt.figure(figsize=(10, 6))
sns.histplot(t_values, bins=100, alpha=0.6, color='g', kde=True)

# Plot VaR and CVaR
plt.axvline(var_percentage/100, color='red', linestyle='--', label=f'VaR ({confi
plt.axvline(cvar_percentage/100, color='purple', linestyle='--', label=f'CVaR ({

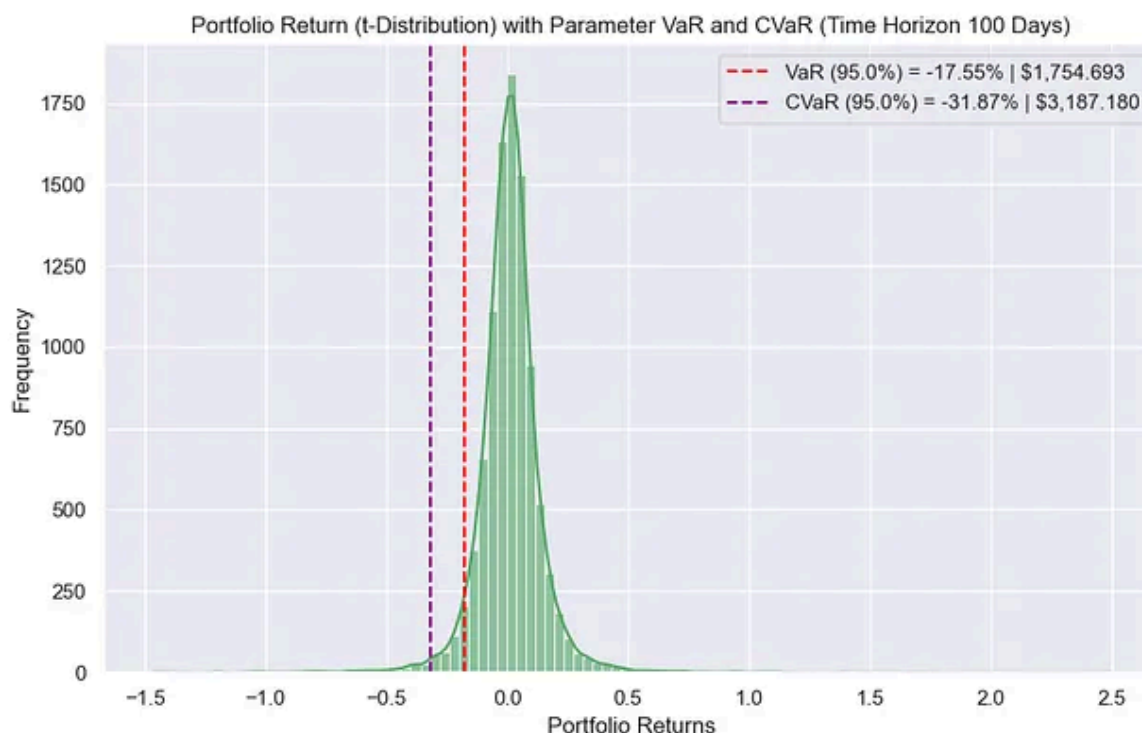
plt.title(f'Portfolio Return (t-Distribution) with Parameter VaR and CVaR (Time
plt.xlabel('Portfolio Returns')
plt.ylabel('Frequency')
plt.legend()
plt.savefig('para_var_cvar.png')
plt.show()

```

```

--- Risk Metrics Summary (Time Horizon: 100 Days, Investment: $10,000) ---
VaR Monetary Loss over 100 days: $1754.69 (-17.55%)
CVaR Monetary Loss over 100 days: $3187.18 (-31.87%)

```



Monte Carlo Simulation Approach using t-distribution

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

def monte_carlo_simulation(initial_value, portfolio_returns, num_simulations, time_horizon):
    # Generate random returns from t-distribution
    dfree, mu, std = stats.t.fit(portfolio_returns)
    random_returns = stats.t.rvs(df=dfree,
                                loc=mu,
                                scale=std,
                                size=(num_simulations, time_horizon))

    # Calculate cumulative returns
    cumulative_returns = np.cumprod(1 + random_returns, axis=1)

    # Calculate portfolio values
    portfolio_values = initial_value * cumulative_returns

    return portfolio_values

def calculate_var_cvar(portfolio_values, confidence_level):
```

```

final_values = portfolio_values[:, -1]
sorted_values = np.sort(final_values)

var_index = int(len(sorted_values) * (1 - confidence_level))
var = initial_value - sorted_values[var_index]

cvar = initial_value - np.mean(sorted_values[:var_index])

return var, cvar

def calculate_var_cvar_percentage(portfolio_values, confidence_level):
    final_returns = (portfolio_values[:, -1] - portfolio_values[:, 0]) / portfolio_values[:, 0]
    sorted_returns = np.sort(final_returns)

    var_index = int(len(sorted_returns) * (1 - confidence_level))
    var_percentage = -sorted_returns[var_index]

    cvar_percentage = -np.mean(sorted_returns[:var_index])

    return var_percentage, cvar_percentage

# Set parameters
initial_value = 10000 # Initial portfolio value
time_horizon = 100 # Number of trading days
num_simulations = 10000 # Number of Monte Carlo simulations
confidence_level = 0.95 # Confidence level for VaR and CVaR

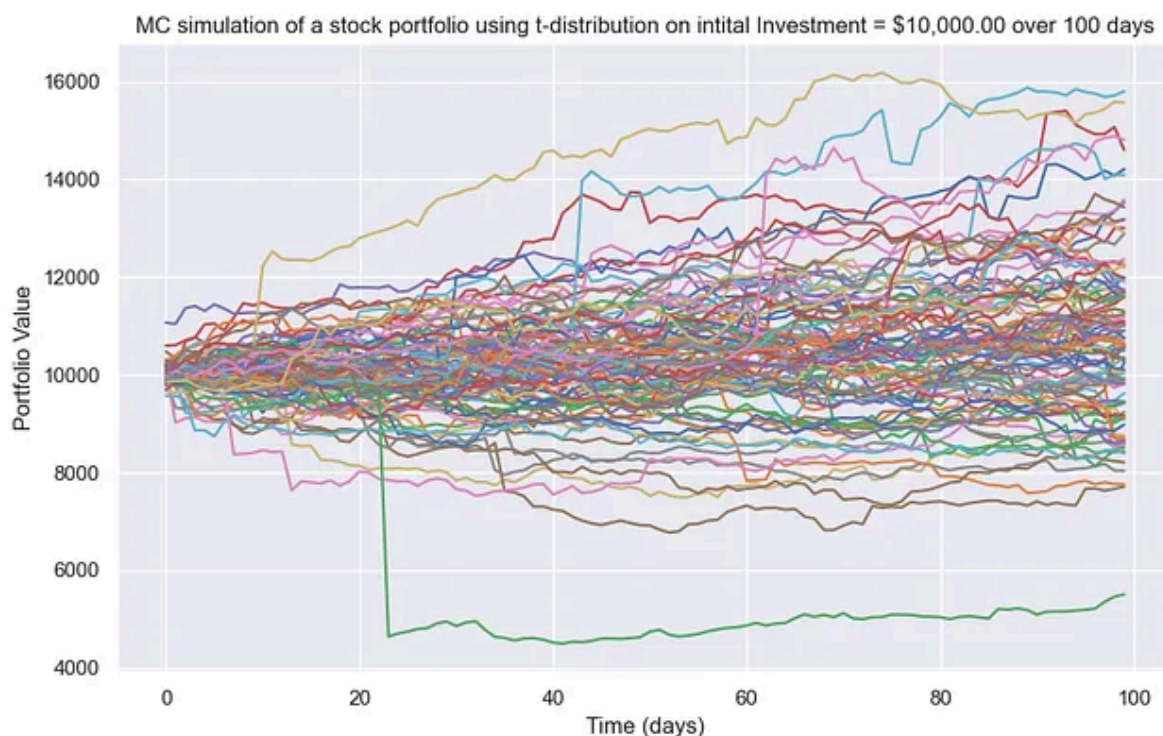
# Run Monte Carlo simulation
portfolio_values = monte_carlo_simulation(initial_value, returns['portfolio'], num_simulations, time_horizon)

# Calculate VaR and CVaR
var, cvar = calculate_var_cvar(portfolio_values, confidence_level)
var_percentage, cvar_percentage = calculate_var_cvar_percentage(portfolio_values, confidence_level)

# Print results
print(f"Value at Risk (VaR) {var_percentage:.2%} at {confidence_level*100}% confidence level")
print(f"Conditional Value at Risk (CVaR) {cvar_percentage:.2%} at {confidence_level*100}% confidence level")

# Plot some sample paths
plt.figure(figsize=(10, 6))
for i in range(100): # Plot 100 sample paths
    plt.plot(portfolio_values[i])
plt.title("MC simulation of a stock portfolio using t-distribution on initial value")
plt.xlabel("Time (days)")
plt.ylabel("Portfolio Value")
plt.savefig('mc_path.png')
plt.show()

```



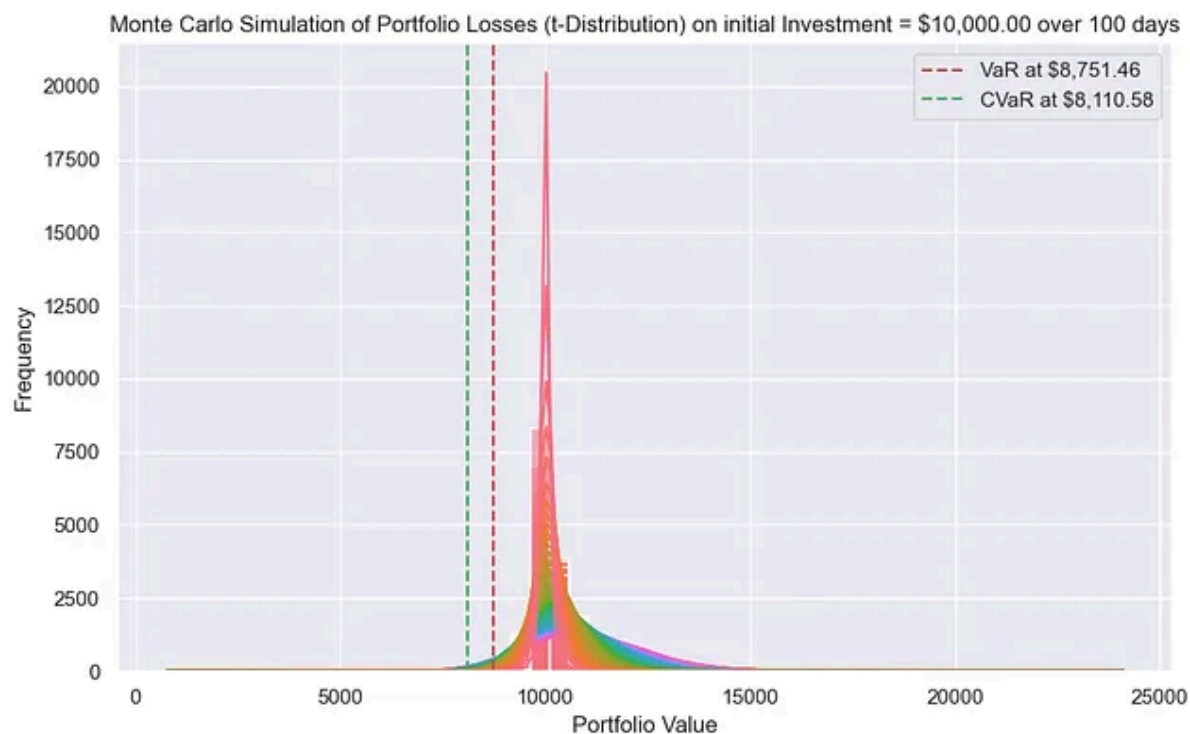
Value at Risk (VaR) 12.49% at 95.0% confidence: \$1,263.77
 Conditional Value at Risk (CVaR) 18.89% at 95.0% confidence: \$1,888.06

```
# Calculate monetary loss
monetary_VaR = initial_value * (1 - var_percentage)
monetary_CVaR = initial_value * (1 - cvar_percentage)
print(f"VaR (Monetary Loss at 95% Confidence): ${var:,.2f}")
print(f"CVaR (Average Loss beyond VaR): ${cvar:,.2f}")
```

VaR (Monetary Loss at 95% Confidence): \$1,263.77
 CVaR (Average Loss beyond VaR): \$1,888.06

```
# Plot histogram of portfolio values
plt.figure(figsize=(10, 6))
sns.histplot(portfolio_values, bins=50, kde=True, color='skyblue')
plt.axvline(monetary_VaR, color='r', linestyle='dashed', label=f'VaR at ${moneta
```

```
plt.axvline(monetary_CVaR, color='g', linestyle='dashed', label=f'CVaR at ${mone
plt.title(f'Monte Carlo Simulation of Portfolio Losses (t-Distribution) on initi
plt.xlabel('Portfolio Value')
plt.ylabel('Frequency')
plt.legend()
plt.savefig('mc_var_cvar.png')
plt.show()
```



Comparison of each VaR & CVaR methods

Historical VaR : 18.38%

Historical CVaR: 32.43%

Parametric VaR : 17.55%

Parametric CVaR: 31.87%

Monte Carlo VaR: 12.49%

Monte Carlo CVaR: 18.89%

Overall, the historical approach has the highest VaR and CVaR, while the Monte Carlo approach has the lowest VaR and CVaR.

Thank you for reading.

Other articles about this topic:

[Managing Risk in Investment Portfolios: A Guide to Calculating VAR and CVAR](#)

[Portfolio VaR and CVaR](#)

[VaR Vs CVaR: Comparing Risk Metrics in Financial Analysis](#)

[Historical Value at Risk \(VaR\) with Python \(YouTube\)](#)

[Parametric VaR and CVaR with Python \(YouTube\)](#)

[Monte Carlo Simulation with value at risk \(VaR\) and conditional value at risk \(CVaR\) in Python \(YouTube\)](#)



Written by Simon Leung

229 followers · 4 following

Edit profile

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.
